
Development of a Software Development Platform

Ina N. Aeon

GitHub Repository

https://github.com/newcastleuniversity-computing/CSC3131_20252026_c2027219

Scope Requirements

The scope outlined a project development platform intended to foster DevOps culture and practices. There are design requirements that focus on the longevity of the product through good maintenance, scalability, and reproducible deployment capabilities. Security must also be carefully considered when designing the architecture. The platform must also integrate features into the workflow, such as a CI pipeline that integrates into a project management platform to establish a continuous feedback loop between developers that records and encourages collaboration, whilst also automating monotonous tasks.

Design Philosophies

Open Source Software

Utilising applications with an open source license in a project reduces not only cost but also the amount of required development and software maintenance time. Application code is maintained communally through collaboration and distributed with licenses such as the GNU General Public License. These community contributions result in improved security and transparency whilst also having the flexibility to be modified to fit individual needs through forking the program's source code, or the project could be maintained by internal developers if communal contributions are no longer being made.[9]

Containerisation

Containerisation simplifies system deployment, maintenance, and migration; a containerised system can be deployed to an on-site server maintained by company staff or hosted off-site in the cloud. Although there is an initial learning curve, these reproducible environments provide consistency and scalability by reducing the manual setup required. Resolving an outage becomes a matter of having an available off-site backup to begin hosting the container elsewhere; in the future, this flexibility could be expanded with added off-site redundancy and replication to further reduce any potential loss or downtime. This implementation takes an approach that is suited for vertical scaling, as the system and its files are portable and can be easily migrated onto newer, better hardware. Horizontal scaling is not within the scope and is not heavily considered in the design implementation, as there is no requirement for ultra-high availability on a distributed scale. Docker was chosen as the deployment platform over other options because it provides reliable reproducibility between systems, there is good documentation and community support, and it is also appropriate for the scale of the system compared to a platform such as Kubernetes, which is better suited for large distributed projects that scale horizontally. Docker also has a feature called Compose, which allows for the simple deployment of containers with a single configuration file.[2]

Frontend Design

Development Platform

When implementing an environment for developers to create issues that track progress and create documentation, Redmine was chosen as its built-in features fit the scope requirements well. Redmine has local Git repository integration out of the box; however, it lacks CI pipeline integration. This can be added using the Redmine API, which lets a CI

system communicate with Redmine. The CI system used is Jenkins; it was chosen over something like GitHub Actions because GitHub Actions may eventually incur costs when scaling usage, potentially creating an additional financial overhead for operation.[3]

Backend Design

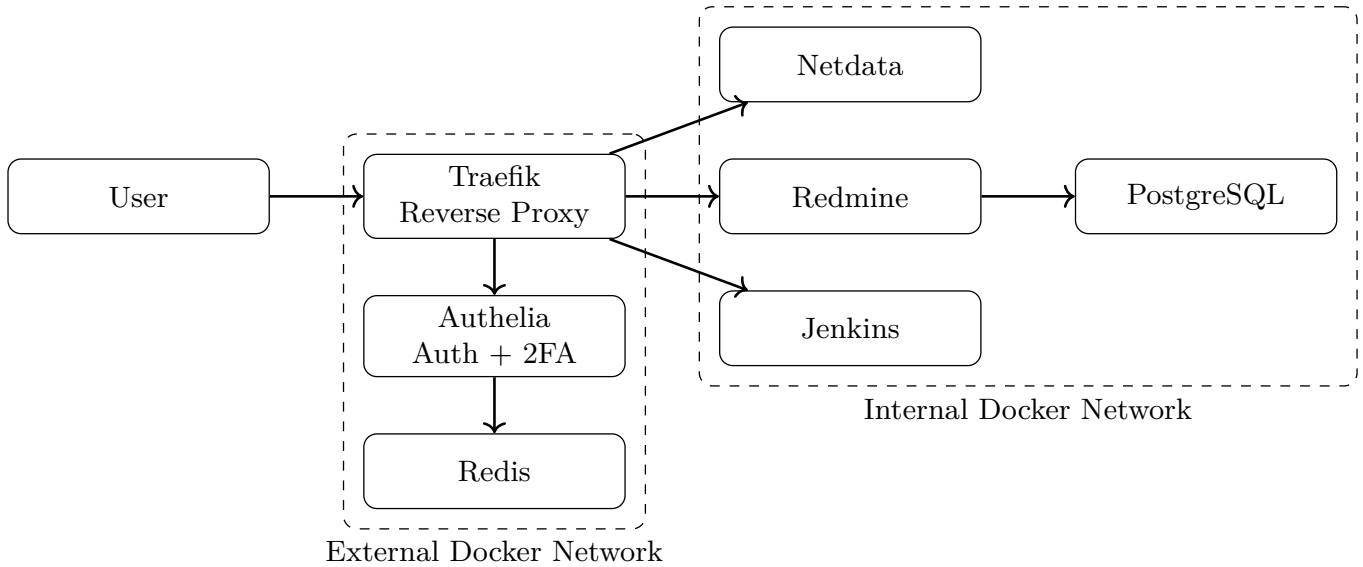


Figure 1: Flow diagram of the proposed system architecture.

Network Isolation

To maintain good security practices, the development system itself and its database are hosted on an internal network to reduce the attack surface. Traefik serves as the bridge between the internal and external networks. This reduces the exposure of internal services by keeping them unreachable until authentication is obtained. Authelia was implemented as the authentication gateway for centralised user access control, and Traefik was chosen as a reverse proxy to sit between the two networks and route traffic securely. Both were chosen because they are open-source and available as Docker containers. Traefik has a simple setup process when compared to using something like Nginx as the reverse proxy.

Data Storage

Two separate database applications were chosen with specific performance goals in mind. Redis is an in-memory database designed for frequent access, making it a good fit for use with Authelia and Traefik, where high availability is critical.[14] PostgreSQL is designed for persistence and data integrity, which makes it better

suited for the development platform itself, so it was chosen for storing data from the Redmine container.[13] Redmine also has a locally bound folder for adding Git repos. This was also an architectural choice designed to securely isolate sensitive internal information from processes on the external network in the event of a security breach.

Monitoring

Netdata was used for process monitoring and system observability. It monitors containers in the external and internal networks. Netdata was specifically chosen for its ease of deployment, customisability, and reliability compared to solutions such as a Grafana-based monitoring stack, which typically requires additional services that create additional points of failure. Each container was given its own health check command, which automatically checks the status of the container. These health check results can be observed in Netdata, allowing an administrator to easily check if each container is functioning correctly. This solution provides comprehensive monitoring across the whole system alongside the monitoring commands included by the Docker runtime.[12]

Implementation

Docker Compose

Creating Docker containers is streamlined through the use of Docker Compose, where defining and deploying containers can be done from a configuration file. It is important to note that when sensitive data, such as passwords and usernames are specified in service configuration, the `.env` file should be used to store that information rather than the compose file itself. This makes it less likely for important login details and tokens to be compromised.

```
ina@puter:~/Documents/University/CSC3131$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
be780da14a50	redmine:5-alpine	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:33696->3000/tcp, :::33696->3000/tcp	redmine
66dd2ce81bdd	authelia/authelia:latest	"/app/entrypoint.sh"	About a minute ago	Up About a minute (healthy)	0.0.0.0:33695->9091/tcp, :::33695->9091/tcp	authelia
733708456a90	netdata/netdata:latest	"/usr/sbin/run.sh"	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:33694->19999/tcp, :::33694->19999/tcp	netdata
80c5cc01a3c2	traefik:v3.1	"/entrypoint.sh --en..."	2 minutes ago	Up 2 minutes	0.0.0.0:80->80/tcp, :::80->80/tcp, 0.0.0.0:443->443/tcp, :::443->443/tcp	traefik
789625a7815e	redis:7-alpine	"docker-entrypoint.s..."	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:33693->6379/tcp, :::33693->6379/tcp	redis
16eac531b43f	postgres:16-alpine	"docker-entrypoint.s..."	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:33692->5432/tcp, :::33692->5432/tcp	postgres
90647a0319f7	jenkins/jenkins:latest	"/usr/bin/tini -- /u..."	2 minutes ago	Up 2 minutes	8080/tcp, 50000/tcp	jenkins

Figure 2: Output of Docker's process listing command, in ideal conditions.

```
ina@puter:~/Documents/University/CSC3131$ sudo docker-compose up -d
Creating network "devplatform_external" with the default driver
Creating network "devplatform_internal" with the default driver
Creating netdata ... done
Creating redis ... done
Creating jenkins ... done
Creating traefik ... done
Creating postgres ... done
Creating redmine ... done
Creating authelia ... done
```

Figure 3: Output of Docker Compose's up command, creating the networks and containers, "-d" can be used to automatically detach from the process and have it run in the background, instead of the whole system going down when the terminal process exits.

Volume and Network Management

Each container has its own volume for storing data and is connected to either the internal and/or external network, which are specified in the compose configuration file. These are then created when Docker Compose is run for the first time (Figures 3, 2). Volumes are stored in the Docker runtime and can be migrated, deleted, or backed up, allowing a future developer to deploy a new system with fresh volumes or an instance with previous volumes attached. Redmine's repository folder is locally bound to allow an administrator to easily clone existing repos or initialize a new repository in the terminal.

Traefik

Host Setup

Setting up a Traefik container in Docker Compose required a handful of labels to configure routing correctly and enforce HTTPS. Ports 80 and 443 were exposed for routing HTTP and HTTPS traffic, and the container was given access to both the internal and external networks to act as the bridge between the two. Traefik only creates routers for containers that are considered healthy by the defined health check, preventing traffic from being routed to unstable services to maintain service resilience. Although not recommended because it would heavily increase the attack surface of the system, this could be exposed to the internet. A future development could add a VPN server as a method to tunnel to the system from the internet.[1]

Middleware Configuration

To be able to route traffic to and from other containers, they must also have labels attached to them in Compose. Each container gets a defined router with its own subdomain. Traefik serves the service if the user is authenticated by

Authelia. For demonstration purposes, the public wildcard DNS resolver "localtest.me" is used. This makes it simple to test solutions without having a domain to obtain a valid certificate with through Let's Encrypt, which otherwise would be used in an implementation that used its own domain.[11][17]

HTTPS Certificate Generation

Let's Encrypt (an open-source certificate authority) can be used to generate a valid HTTPS certificate. It can be invoked in Docker Compose by configuring additional labels in the Traefik container. However, the demonstration system does not do this, and instead, Traefik uses a locally generated certificate. This is acceptable in development and testing environments, but will result in browsers not trusting application pages, usually displaying a warning.[10]

Authelia

Authelia requires configuration.yml to function, which specifies the server's address, session info and local storage location. There is also a dark mode option, which is much better on the eyes. An admin user must also be specified in the users.yml configuration file with a password hashed using the Argon2 password hashing algorithm, which is included with the container. An administrator can create an Argon2 hashed password by interfacing with the container and running a command.

Automatic Redirect

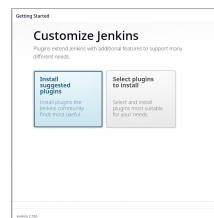
Services can be configured to automatically redirect users who are trying to access a page that they don't currently have access to back to the login screen. This functionality is enabled by specifying a Traefik router rule that interfaces with Authelia authentication middleware.

Jenkins

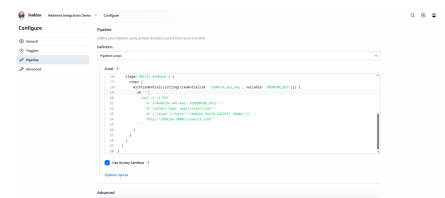
Jenkins is an open-source CI pipeline tool used to validate code changes through automated verification tasks. It is versatile and can be used for validating application builds and system deployment from various languages and deployment solutions, making it a solid choice for implementing a CI pipeline compared to interfacing with a service such as GitHub Actions, which has similar functionality but can potentially introduce a subscription fee at enterprise-level usage.

Setup

Setting up Jenkins, beyond defining the container in Docker Compose, was relatively straightforward. The initial setup requires the administrator to get the randomly generated administrator password from the runtime itself, change it to a new one, or skip and keep the initial password, then specify which packages should be initially installed. For example, validation of Java projects can be done in stages, starting with a build tool such as Maven that either creates an artifact or produces an error. The artifact produced could then be validated further with the results of pre-written unit tests. These multiple highly-configurable stages of automatic testing can heavily reduce the effort of testing and allow developers to have a smoother workflow.



(a) Installing base plug-ins for Jenkins in the second part of the Jenkins setup.



(b) Configuring the pipeline task.

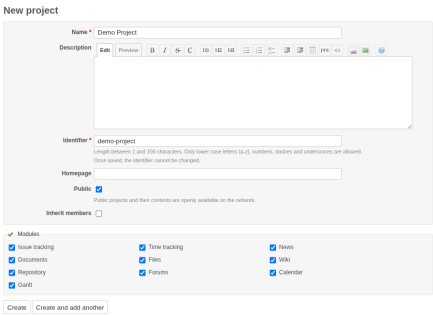
Redmine Integration

Once Jenkins is set up, a pipeline task can be created that pushes data to Redmine's REST API. These tasks are written in the Groovy syntax and use shell commands such as curl to communicate with a service's API. Although this adds additional experience required to develop pipelines, the flexibility of Groovy allows for complex pipelines to be constructed.

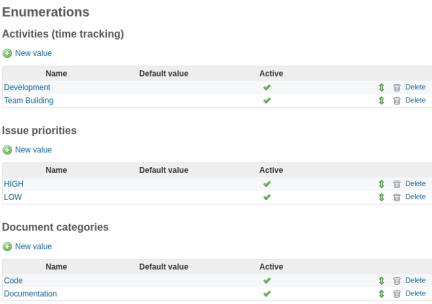
Creating a Build Task

Creating a build task involves using Jenkins' native scripting language. Pipelines can use curl to push an HTTP request (called a PUT request) to the Redmine API, which then comments on an issue.[8][15]

Redmine



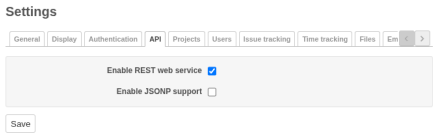
(a) Creating a project in Redmine.



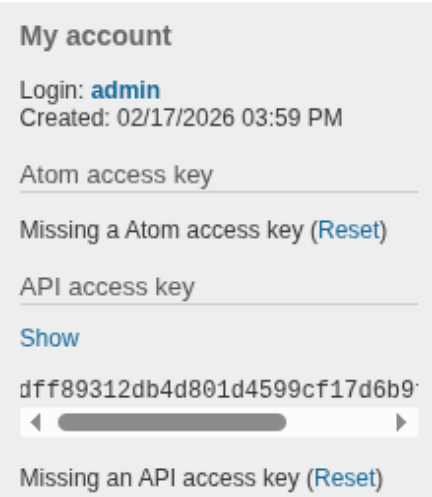
(b) Configuration options for Issue Priorities, Document Categories and Activities in the Enumerations menu.



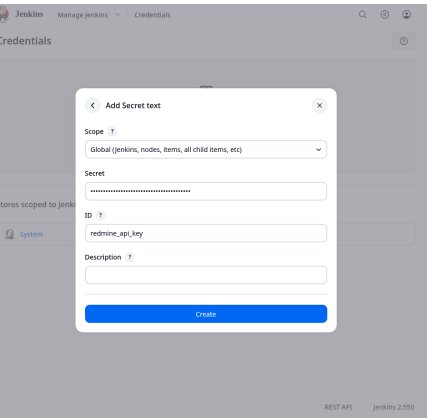
(c) Configuring the statuses that issue trackers can use.



(d) Enabling the REST API in Redmine.



(e) An API Key now appears in the "My Account" section.



(f) Specifying the API Key in the credentials menu of Jenkins.

Setup

Redmine requires some basic setup, such as initially changing the default admin password (the default username is admin, and the default password is admin). Further setup of Redmine's basic functionality involves creating a project and specifying issue trackers, issue statuses, and issue priorities in the Administration menu. (Figures 5a, 5b, 5c)

Features

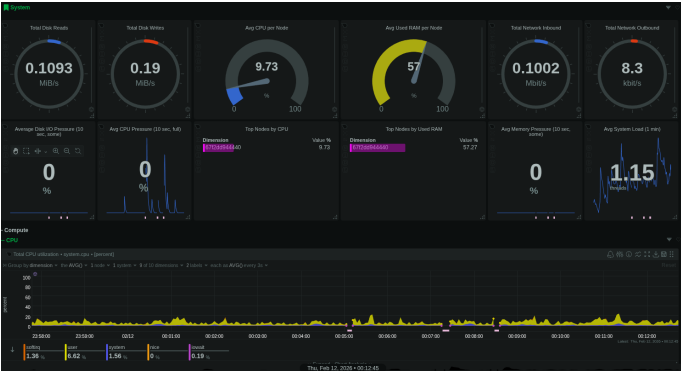
Redmine has various features that are useful for collaborative development. Projects can be created and assigned to users, and those projects contain issues, documentation, files, and development timelines. Project pages with registered repositories also show recent Git commits. These features provide a full suite for not only developer collaboration, but also well-coordinated project development processes.

Jenkins CI Integration

Jenkins integration features can be added to Redmine by exposing the REST API in Redmine and providing an API key as an API credential to Jenkins for use in a pipeline task, allowing integration into the discussion and tracking features provided by Redmine to evaluate action results (Figures 5d, 5e, 5f). This functionality can be used to create

a feedback loop between pipeline progress in Jenkins and project management tracking in Redmine by creating a link between Jenkins’ build results and Redmine’s project issues.[15]

Netdata



(a) Resource usage metrics in Netdata.



(b) Docker metrics in Netdata.

Netdata monitors both system resources and the Docker runtime, providing metrics on system performance and Docker’s health checks (Figure 6a, 6b). These metrics allow for better observability of the system’s health compared to regular Docker health checks, where health is determined by endpoint response rather than resource usage. These checks can overlook services that consume too many resources and do not report a degradation in service health, potentially leading to unexpected disruption. Netdata can prevent this issue through alerts that are triggered in the case of specified conditions. This can reduce potential disruption or downtime by addressing service issues before a failure can occur. These alerts are also useful for coordinated downtime resolution when unavoidable crashes occur. A system administrator monitoring these alerts can be notified almost immediately, which is especially useful when long CI pipeline tasks run by Jenkins can heavily consume system resources and cause system instability.[12]

Cost Analysis

Hardware Requirements

Given the minimum requirements specified by the developers of each application, the minimum storage required for only the containers would be around 20GB. This does not reflect the actual requirements, as storage overhead for the operating system and container software must also be taken into account. The minimum required amount of memory is around 4GB, and the system is relatively lightweight on the CPU. Though requirements for storage, processing, and memory expansion will eventually occur, especially if additional users are added, as data would accumulate in storage over time, and the additional requests of new users would create a growing hardware utilisation footprint.

Initial Investment

The implementation may appear inexpensive, but the system implemented requires a server to be hosted on. There are various options to consider when choosing a host, with their own advantages and drawbacks. Consequently, the highest cost incurred initially and over time is the hosting and maintenance of the system’s infrastructure.

Hosting Options

On-Site Hosting

On-site hosting involves purchasing, maintaining, and securing the hardware that the platform runs on. This is useful for having complete control over the entire system. However, dedicated sysadmins are required for reliable platform

maintenance. This could prove expensive in the case of hardware failure, and any downtime must be resolved by the host itself. This could potentially result in long downtimes or service degradation if there isn't a competent administrator maintaining the system. The host also incurs the electrical bill of running this server; power usage at a large scale can be expensive. This option is only viable if experienced staff are available at short notice and there is plenty of financial overhead for hardware replacement and utilities upkeep.

Infrastructure as a Service (IaaS)

Off-site (Infrastructure-as-a-Service) hosting delegates on-site hardware maintenance to a cloud provider and provides the server infrastructure itself for an administrator to host their software on, removing the need to maintain on-site servers. Companies such as Hetzner and Scaleway offer this infrastructure as a product in the form of Virtual Private Servers that run a specified OS (commonly Ubuntu), with the subscription cost outlining the system hardware itself, not each user, making it a much more affordable and scalable option compared to a SaaS solution (Table 1), eliminating vendor lock-in and upfront hardware costs.

Service	Product	CPU	RAM	Storage	Bandwidth/Traffic	Price
Scaleway	Start-1-L	4C/4T	16 GB	2x 1 TB HDD	200 Mbps	EUR 19.99/mo
Hetzner	Cloud CPX42	8 vCPU	16 GB	320 GB SSD	20 TB	EUR 19.99/mo
Hostinger	KVM 4	4 vCPU	16 GB	200 GB SSD	16 TB	GBP 29.99/mo

Table 1: Example hosting options.[16][6][7]

Sustainability

Over time, storage requirements would increase with the amount of accumulated data, and CPU and RAM requirements could increase with how many concurrent users the system has. It would not only reduce the intensity of maintenance but would also allow for cheaper scaling in the future. IaaS hosting offers "infinite" scalability; though in reality, this could become expensive over time as running costs would increase with resource usage, especially storage, where options may be limited to NVMe Storage, which can be expensive. The ideal solution, cost-wise and for scalability, would be to host this system using an IaaS solution. Migrating from one IaaS host to another for better pricing or scalability using the solution implemented would be simple, as the system can be exported as a disk image and moved elsewhere.

Risk and Responsibilities

Due to the responsibilities of the hardware and service maintenance lying with the provider itself, issues on the provider's end could result in a loss of accessibility or even sensitive data. This risk could be mitigated through external backups of the data hosted on the service. Though reputable providers have their own fail-safes and will usually have little to no issue with maintaining these responsibilities, it is still important to ensure data integrity in the event of a loss.

Comparison to GitLab

Software as a Service (SaaS)

Using a system hosted by a company as a service would eliminate the need to maintain the hardware and service with a serverless architecture approach, but would restrict the platform's capabilities to what the provider offers.

Sustainability

There is also usually a higher cost to using a SaaS solution, as most charge per-seat (user). This type of subscription model would increase financial overhead significantly over time if the company were to scale, which then would exacerbate an issue called vendor lock-in, where it may prove difficult and disruptive to move away from the platform.

Scope Compatibility

GitLab has both a SaaS solution and a self-hosted solution that meet the scope requirements of the project, with documentation and project management applications similar to Redmine and its own CI pipeline similar to Jenkins.

Reputation

GitLab has a good track record, except for a major incident in 2017 where six hours of data were lost when a developer ran commands in the incorrect terminal window. This caused major disruption but was eventually rectified and service was restored.[4]

Maintenance

GitLab's SaaS solution is not only expensive when compared to using their self-hosted solution on a VPS, but could also result in expensive vendor lock-in and strenuous migration. The lowest tier of the subscription service costs £25 per user per month.[5] The self-hosted version would definitely be cheaper, though it would still require a host from either the administrator or an IaaS provider, but would provide a solid project management platform and CI pipeline.

Conclusion

The solution designed and implemented meets the scope requirements, providing a full development and project management suite, a CI pipeline, and Git repository integration using Redmine and Jenkins. The backend also includes good monitoring and observability, with robust security practices that reduce the attack surface by isolating externally facing services from confidential internal services, using Authelia's user access control as an authentication gateway and Traefik as a reverse proxy. This solution provides solid foundations for a well-integrated project development platform but still leaves room for further development. Improved Jenkins pipeline tasks could be implemented to deepen Redmine API integration, such as sending build results to issues only within a specific category by sending GET requests with curl to obtain a list of categorized issues. The additional Minecraft server plugin demonstration highlighted the potential to add more automation to the deployment stage. Better observability could also be achieved by expanding the functionality of Netdata's threshold alerts by configuring notifications that are sent to a sysadmin via email or trigger a webhook to alert an administrator through a messaging platform such as Discord.

References

- [1] DigitalOcean. *How To Use Traefik v2 as a Reverse Proxy for Docker Containers on Ubuntu 20.04*. 2020. URL: <https://www.digitalocean.com/community/tutorials/how-to-use-traefik-v2-as-a-reverse-proxy-for-docker-containers-on-ubuntu-20-04>.
- [2] Docker Inc. *Docker Documentation*. 2024. URL: <https://docs.docker.com/>.
- [3] GitHub. *GitHub Actions Documentation*. 2024. URL: <https://docs.github.com/en/actions>.
- [4] GitLab. *Database Incident of January 31, 2017*. 2017. URL: <https://about.gitlab.com/blog/2017/02/01/gitlab-dot-com-database-incident/>.
- [5] GitLab. *GitLab Pricing*. 2026. URL: <https://about.gitlab.com/pricing/>.
- [6] Hetzner. *Hetzner Cloud*. Accessed: 2026-02-19. n/a. URL: <https://www.hetzner.com/cloud/>.
- [7] Hostinger. *VPS Hosting Pricing (Plan Selector)*. Accessed: 2026-02-19. n/a. URL: <https://www.hostinger.com/uk/pricing/vps-hosting#plan-selector>.
- [8] Jenkins Project. *Getting started with Pipeline*. n/a. URL: <https://www.jenkins.io/doc/book/pipeline/getting-started/>.
- [9] Gene Kim et al. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution, 2016.
- [10] Let's Encrypt. *How It Works*. 2025. URL: <https://letsencrypt.org/how-it-works/>.

-
- [11] localtest-dot-me. *localtest.me (github)*. n/a. URL: <https://github.com/localtest-dot-me/localtest-dot-me.github.com>.
 - [12] Netdata. *Netdata Documentation*. 2024. URL: <https://learn.netdata.cloud/>.
 - [13] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/>.
 - [14] Redis Ltd. *Redis Documentation*. 2024. URL: <https://redis.io/docs/>.
 - [15] Redmine Project. *REST API – Redmine*. Redmine. n/a. URL: https://www.redmine.org/projects/redmine/wiki/Rest_api.
 - [16] Scaleway. *Dedibox Start*. Accessed: 2026-02-19. n/a. URL: <https://www.scaleway.com/en/dedibox/start/>.
 - [17] Traefik Labs. *Traefik Documentation*. 2024. URL: <https://doc.traefik.io/traefik/>.